Object Oriented Programming – CS/CE 224/272

Nadia Nasir

Habib University, Fall 2025

Class Methods

Functions associated with a class are declared in one of two ways:

`ReturnType FuncName(params) { code }`

function is both declared and defined (code provided)

`ReturnType FuncName(params);`

function is merely declared, we must still define the body of the function separately

To call a method we use the . form:

classinstance.*FuncName*(*args*);

FuncName is a field just like any other field in the structured variable *classinstance*

Defined Methods

The function getX is defined as part of class Robot

```
class Robot {
  public:
    float getX() { return locX; }
};
Robot r1;
```

Calling a class member function using . operator

```
cout << r1.getX() << endl; // printsr1's locX
```

Defining Methods Separately

For methods that are declared but not defined in the class we need to provide a separate definition

To define the method, you define it as any other function, except that the name of the function is *ClassName*::*FuncName*

:: is the scope resolution operator, it allows us to refer to parts of a class or structure

A Simple Class

```
class Robot {
public:
    void setLocation(float x, float y);
private:
    float locX;
    float locY;
    float facing;
};

void Robot::setLocation(float x, float y) {
if ((x < 0.0) || (y < 0.0))
    cout << "Illegal location!!" << endl;
else {
    locX = x;
    locY = y;
}
}
```

Referring to Class Fields

What are locX and locY in setLocation??

Since a class function is always called on a class instance, locX, locY are those fields of that instance

Example:

```
Robot r1;
Robot r2;
r1.setLocation(5,5);  // locX is from r1
r2.setLocation(3,3);  // locY is from r2
```

```
void Robot::setLocation(float x, float y) {
  if ((x < 0.0) || (y < 0.0))
    cout << "Illegal location!!" << endl;

else {
    locX = x;
    locY = y;
  }
}
```

Types of Class Methods

Generally we group class methods into three broad categories:

accessors - allow us to access the fields of a class instance (examples: getX, getY, getFacing), accessors do not change the fields of a class instance

mutators - allow us to change the fields of a class instance (examples: setLocation, setFacing), mutators do change the fields of a class instance

manager functions - specific functions (constructors, destructors) that deal with initializing and destroying class instances (more in a bit)

Accessors and Mutators

Why do we bother with accessors and mutators?

Why provide getX()?? Why not just make the locX field publicly available?

By restricting access using accessors and mutators we make it possible to change the underlying details regarding a class without changing how people who use that class interact with the class

Example: what if I changed the robot representation so that we no longer store locX and locY, instead we use polar coordinates (distance and angle to location)?

- If locX is a publicly available field that people have been accessing it, removing it will affect their code
- If users interact using only accessors and mutators then we can change things without affecting their code

Rewritten Robot Class

```
class Robot {
  public:
    float getX() { return dist * cos(ang); }
    float getY() { return dist * sin(ang); }
    void setLocation(float x, float y);
  private:
    float locX;
    float locY;
};
void Robot::setLocation(float x, float y) {
  if ((x < 0.0) || (y < 0.0))
    cout << "Illegal location!!" << endl;
  else {
    locX = x;
    locY = y;
  }
}
```

Object-Oriented Idea

Declare implementation-specific fields of class to be private
Provide public accessor and mutator methods that allow other users to connect to the fields of the class

- often provide an accessor and mutator for each instance variable that lets you get the current value of that variable and set the current value of that variable
- to change implementation, make sure accessors and mutators still provide users what they expect

The Need for Manager Functions

To make the previous program work we have to explicitly call the routine initializeRobot, wouldn't it be nice to have a routine that is automatically used to initialize an instance -- C++ does

C++ provides for the definition of manager functions to deal with certain situations:

- constructors (ctor) - used to set up new instances

  default - called for a basic instance
  copy - called when a copy of an instance is made (assignment)
  other - for other construction situations

- destructors (dtor) - used when an instance is removed

When Managers Called

```
class Robot {
public:
    float getX() ;
    float getY() ;
    void setLocation(float x, float y);
private:
    float locX, locY , facing;
};


int main() {
Robot r1;  // default ctor (on r1)
Robot r2;  // default ctor (on r2)
Robot* r3ptr;

r3ptr = new Robot; // default ctor on (new) Robot object r3ptr points to
r2 = r1;  // copy ctor
delete r3ptr; // dtor
} // dtor (on r1 and r2) when main ends
```

The Default Constructor (ctor)

A default ctor is a ctor that either,

- has **no parameters**, or
- if it has parameters, **all the parameters have default values**.

```
class Robot {
public:
    float getX() ;
    float getY() ;
    void setLocation(float x, float y);
    Robot() {
            locX = 0.0;
            locY = 0.0;
            facing = 3.1415 / 2;
            }

private:
    float locX, locY , facing;
};
```

Parameterized ctors

Constructors can be overloaded.
So you can have multiple ctors will varying number of parameters.

Similarly, you can have a single ctor, with three parameters, all with default arguments.

```
class Robot {
public:
    float getX() ;
    float getY() ;
    void setLocation(float x, float y);
   Robot(float x, float y, float face) {
                        locX = x;
                        locY = y;
                        facing = face;
   }
private:
    float locX, locY , facing;
   };


int main() {
    Robot r1(5.0,5.0,1.5);
    Robot r2(5.0,10.0,0.0);
    Robot* rptr;
    rptr = new Robot(10.0,5.0,-1.5);

    return 0 ;
}
```

A Combination ctor

Can combine a ctor that requires arguments with the default ctor using default values:

```
class Robot {
public:
    float getX() ;
    float getY() ;
    void setLocation(float x, float y);
    Robot(float x = 0.0, float y = 0.0, float face = 1.57075)
    {
            locX = x; locY = y; facing = face;
    }

private:
    float locX, locY , facing;
   };
int main() {
    Robot r1;   // ctor called with default args
    Robot r2(); // ctor called with default args
    Robot r3(5.0); // ctor called with x = 5.0
    Robot r4(5.0,5.0); // ctor called with x,y = 5.0
    return 0 ;
}
```

Hiding the Default ctor

Sometimes we want to make sure that the user gives initial values for some fields, and we don’t want them to use the default ctor

To accomplish this we declare an appropriate ctor to be used in creating instances of the class in the public area, then we put the default ctor in the private area (where it cannot be called)

Example ctor

```
class Robot {
  public:
    Robot(float x, float y, float face) {
      locX = x;
      locY = y;
      facing = face;
    }
  private:
    Robot() {}
}
```

calling:

```
  Robot r1(5.0,5.0,1.5);
  Robot r2;  // ERROR, attempts to call default ctor
```

What is a Constructor (ctor)?

Compiler searches for an accessible constructor that is a **match** for the initialization values provided by the caller.

If an accessible matching constructor is found,
memory for the object is allocated,
then the constructor function is called.

If no accessible matching constructor can be found,
Compiler error.

What is a Constructor (ctor)? (Self-Study)

Key insight

Many new programmers are confused about whether constructors create the objects or not. They do not -- the compiler sets up the memory allocation for the object prior to the constructor call. The constructor is then called on the uninitialized object.

However, if a matching constructor cannot be found for a set of initializers, the compiler will error. So while constructors don't create objects, the lack of a matching constructor will prevent creation of an object.

What is a Constructor (ctor)? (Self-Study)

Beyond determining how an object may be created, constructors generally perform two functions:

- They typically perform initialization of any member variables (via a member initialization list)
- They may perform other setup functions (via statements in the body of the constructor). This might include things such as error checking the initialization values, opening a file or database, etc...

After the constructor finishes executing, we say that the object has been "constructed", and the object should now be in a consistent, usable state.

What is a Constructor (ctor)?

Unlike normal member functions, constructors have specific rules for how they must be named:

- Constructors must have the same name as the class (with the same capitalization). For template classes, this name excludes the template parameters.
- Constructors have no return type (not even `void`).

Constructors are usually **public** members.

Destructors (dtors)

Will this work?

No, because you don't have a default ctor *(you didn't define it and the compiler will not provide it)*.

Therefore, we definitely need to provide an initializing value for our object (size in this e.g.).

```
class MyList
{
private:
    double* m_array;
    int m_length;

public:
    int getLength() const { return m_length; }

    MyList(int length)
        : m_array{ new double[length]{} },
          m_length{ length }
    {}
};
```

```
int main()
{
    MyList list1;

    return 0;
}
```

We now have a **double** array of five elements **on the heap**.

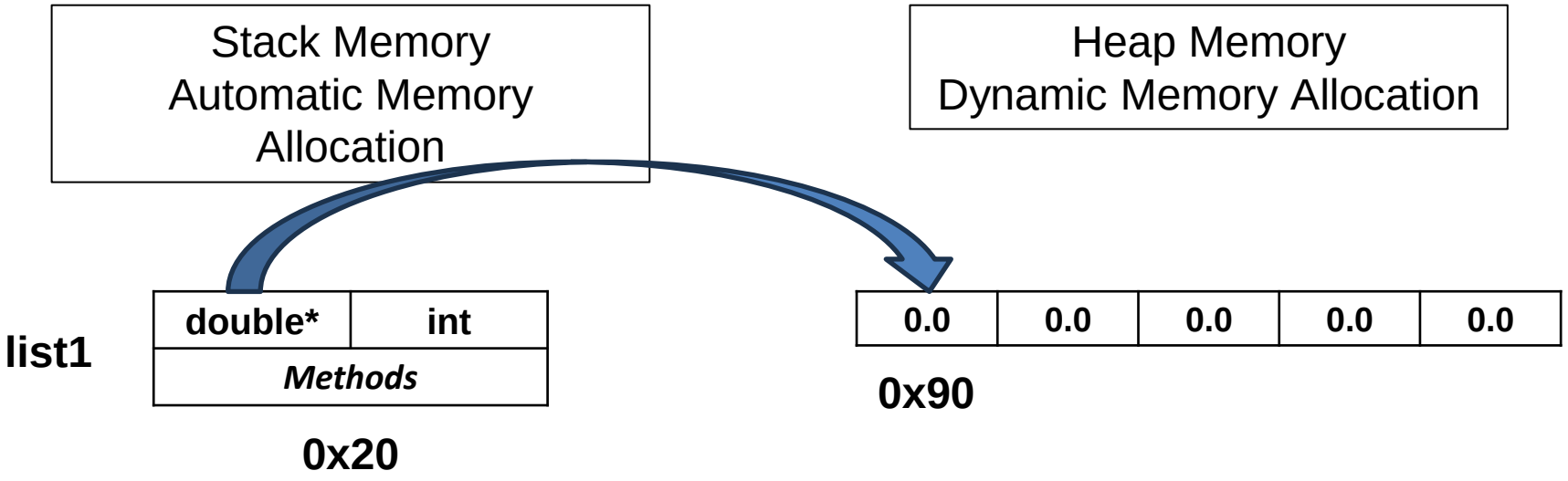
```
class MyList
{
private:
    double* m_array;
    int m_length;

public:
    int getLength() const { return m_length; }

    MyList(int length)
        : m_array{ new double[length]{} },
          m_length{ length }
    {}
};
```

```
int main()
{
    MyList list1{ 5 };

    return 0;
}
```



At the end of **main(), list1** will go out of scope and will get **automatically deallocated**. What will happen to the array?

MEMORY LEAK!

Where can we deallocate the memory?

We can do it in a destructor. Note the syntax.

```
class MyList
{
private:
    double* m_array;
    int m_length;

public:
    int getLength() const { return m_length; }

    MyList(int length)
        : m_array{ new double[length]{} },
          m_length{ length }
    {}

    ~MyList()
    {
        delete[] m_array;
    }
};
```

```
int main()
{
    MyList list1{ 5 };

    return 0;
}
```

Destructors

Whenever an object is about to be destroyed, a destructor is called.

- Whenever an object is going out of scope (if it's on the stack).
- Whenever you **delete** an object (if it's on the heap).

Destructor, like the constructor, is a special method.

Destructors are designed to allow a class to do any necessary clean up before an object of the class is destroyed.

- In our case, it is deallocating the reserved heap memory.

Destructors

Like constructors, destructors have specific naming rules:

1. The destructor must have the same name as the class, preceded by a tilde (~).
2. The destructor can not take arguments.
3. The destructor has no return type.

A class can only have a single destructor.

Destructors may safely call other member functions since the object isn't destroyed until after the destructor executes.

Note

- *Your entire class **must** have,*
  - *one dtor (implicit or explicit), and,*
  - *at least one ctor (implicit or explicit).*
- *Destructor overloading, therefore, is **not** a thing.*

Dtors are implicitly provided by the compiler if you do not explicitly write it.
If you are not doing any cleanup, it is fine to rely on the implicit dtor.

```
class MyList
{
private:
    double* m_array;
    int m_length;

public:
    int getLength() const { return m_length; }

    MyList(int length)
        : m_array{ new double[length]{} },
          m_length{ length }
    {}

    ~MyList()
    {
        delete[] m_array;
    }
};
```

```
int main()
{
    MyList list1{ 5 };

    return 0;
}
```